

'Terraform

Common commands:

<code>apply</code>	Builds or changes infrastructure
<code>console</code>	Interactive console for Terraform interpolations
<code>destroy</code>	Destroy Terraform-managed infrastructure
<code>env</code>	Workspace management
<code>fmt</code>	Rewrites config files to canonical format
<code>get</code>	Download and install modules for the configuration
<code>graph</code>	Create a visual graph of Terraform resources
<code>import</code>	Import existing infrastructure into Terraform
<code>init</code>	Initialize a Terraform working directory
<code>login</code>	Obtain and save credentials for a remote host
<code>logout</code>	Remove locally-stored credentials for a remote host
<code>output</code>	Read an output from a state file
<code>plan</code>	Generate and show an execution plan
<code>providers</code>	Prints a tree of the providers used in the configuration
<code>refresh</code>	Update local state file against real resources
<code>show</code>	Inspect Terraform state or plan
<code>taint</code>	Manually mark a resource for recreation
<code>untaint</code>	Manually unmark a resource as tainted
<code>validate</code>	Validates the Terraform files
<code>version</code>	Prints the Terraform version
<code>workspace</code>	Workspace management

1. Terraform CLI Help

The command is **terraform <commandname> – help**

2. Show Version Command

- **terraform version:** to show the latest version of Terraform

3. Initialize Infrastructure Commands

- **terraform init:** It commands let the tool initiate the working directory.
- **terraform init -input=true:** It is the command to ask for a valid input if the code demands it.
- **terraform init -lock=false:** It is the command that is run for disabling the locking of state files during the dedicated operations.

4. Get Commands

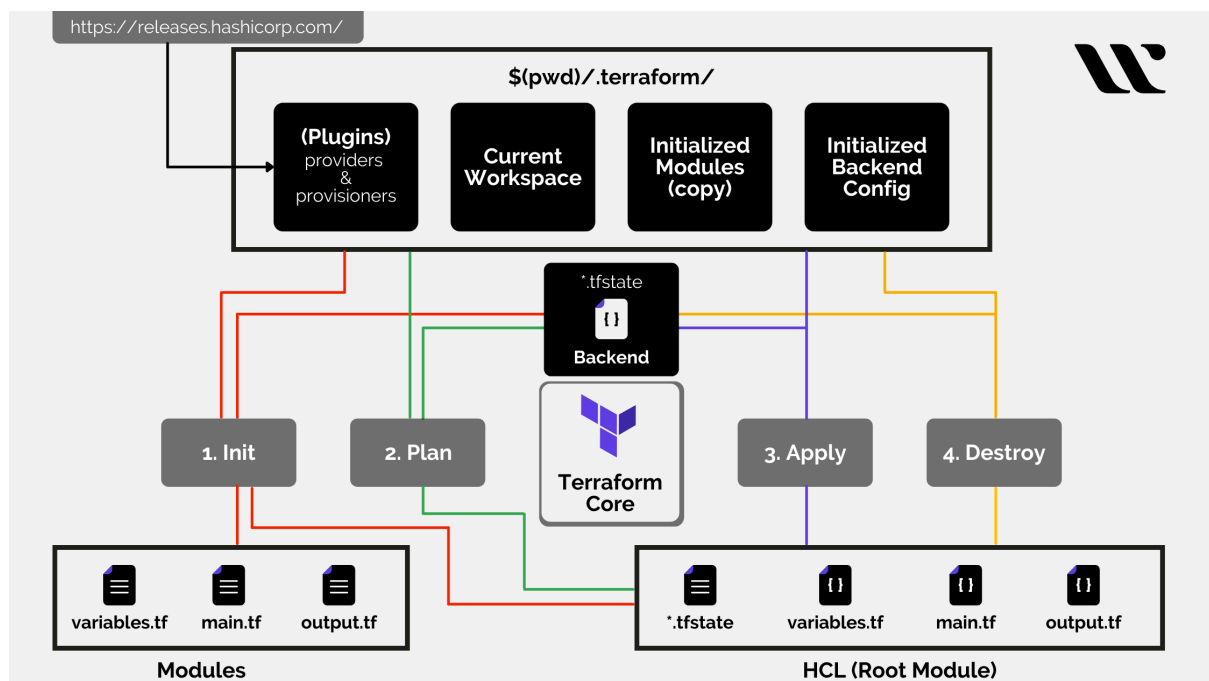
- **terraform get:** It is the command to allow the tool for downloading and updating the existing modules within the root.
- **Terraform get -update=true:** It is the command under which the downloaded modules will be checked for available updates. If found, then those modules will be updated.

5. Provision Infrastructure Commands

- **terraform plan:** It is the command that works on creating an execution plan and implementing a DRY RUN.
- **terraform plan -out=path:** This command saves the created plan in the form of a file.
- **Terraform plan-destroy:** This command derives a destroy plan for the select resources.
- **Terraform apply:** This command is meant to implement the required changes to the existing infrastructure.
- **Terraform apply- auto-approve:** With this command, the changes made to the infrastructure will be implemented without any prompts.
- **Terraform apply -refresh=true:** This command is meant to update the resource states before the execution of the plan and applying the changes.
- **Terraform apply -input=false:** It is meant to request input of variables if they are not directly set.
- **Terraform apply -var 'foo=bar':** This command allows to set a multi-use variable within the configuration.
- **Terraform apply -var-file=foo:** This command is called for specifying files consisting of key pairs for the values that are variable.
- **Terraform apply -target:** This command is meant to apply or deploy the planned changes to the selected resources.
- **Terraform destroy -auto-approve:** With this command, the resource will be deleted or destroyed without any prompts.
- **Terraform destroy -target:** With this command, you can target specific resources or dependencies that are meant to be destroyed.

6. Terraform Workspace Commands

- **Terraform workspace new:** This command will allow you to create a new workspace and select it for further operations.
- **Terraform workspace select:** This command will allow you to select the existing workspace.
- **Terraform workspace list:** This command will display the list of all the existing workspace.
- **Terraform workspace show:** With this command, the name of the current workspace will be shown on the screen.
- **Terraform workspace delete:** It is meant to delete any empty workspace that you will target.



7. Format and Validate Terraform Command or Code

- **Terraform fmt:** This is the formatting code based upon the HCL canonical standard.
- **Terraform validate:** With this command, you get the potential to validate the configuration files for their use in the form of syntax.
-

8. Inspect Infrastructure Commands

- **Terraform graph:** This command creates a graph listing all resources within the configuration.
- **Terraform output:** This command is to list the root module outputs.

- **Terraform output instance_public_ip:** With this command, you can call for a listing of any specific output.
- **Terraform output -json:** It is meant to list the outputs, particularly in JSON formatting.
- **Terraform show:** This command can access the plan or state file for providing a human-readable or understandable output.

9. Terraform Import Commands

Terraform import aws_instance.foo i-xyz123: This command is meant to import an AWS instance of a specific ID. In this sample command, the ID is i-xyz123. It will be imported onto the aws_instance resource that is named as 'foo'.

Try HashiCorp Certified Terraform Associate certification [Practice Tests](#) here before attempting the real exam!

10. Terraform State Manipulation Commands

- **Terraform state list:** It is meant to display all the resources in the form of a list within the state file.
- **Terraform state list aws_instance.my_ec2:** The resource name that you infuse within this command will be the only one to be listed.
- **Terraform state mv:** This command can be used for moving any specific item to the state file.
- **Terraform state rm:** This command is meant to remove the items from the select state file.
- **Terraform state pull:** This command will pull the present state file and output to stdout.
- **Terraform state push:** This command is meant to update any remote state from any of the local state files.
- **Terraform state show aws_instance.my_ec2:** It is meant to show the detailed attributes of any of the single resources.
- **Terraform state refresh:** It is meant to refresh the state file.

Read more on [Terraform vs CloudFormation vs Ansible](#) today!

11. Terraform Miscellaneous Commands

- **Echo "1+5"| terraform console:** This command is used to display a predicted result of any of the expressions as an output.
- **Terraform taint aws_instance.my_ec2:** It marks a particular resource as tainted. As a result of which it is forced to destroy and be recreated in the next APPLY run.

- **Terraform untaint aws_instance.my_ec2:** The tainted mark upon the resource is removed with this command.
- **Terraform force-unlock LOCK_ID:** The lock upon the state file of the ongoing configuration is removed.
- **Terraform login:** This command will get and save API tokens from the cloud.
- **Terraform logout:** This command will remove all the stored credentials during the login process.

CLI commands

- Initialize Terraform: `terraform init`
- Create a plan: `terraform plan`
- Apply the plan: `terraform apply`
- Destroy the infrastructure: `terraform destroy`
- To import an existing infrastructure: `terraform import [resource_name] [resource_id]`
- To see the state of the infrastructure: `terraform show`
- To list all the resources that Terraform is managing: `terraform state list`
- To get detailed information about a specific resource: `terraform state show [resource_name]`
- To validate the syntax of the Terraform files: `terraform validate`
- To see a list of all the available Terraform providers: `terraform providers`
- To see the version of Terraform currently installed: `terraform version`
- To override a variable value from the command line: `terraform apply -var '[variable_name]=[value]'`

TF file language

- To create a new Terraform module: `terraform module new [module_name]`
- To use a module in the Terraform configuration: `module "[module_name]" {}`
- To pass variables to a module: `module "[module_name]" { [variable_name] = [variable_value] }`
- To use a different version of a provider: `provider "[provider_name]" { version = "[version]" }`
- To create and edit a Terraform variable file: `terraform.tfvars`
- To reference a variable in the Terraform configuration: `var.[variable_name]`

Worth noting

- Before running the `terraform apply` command, check the plan using `terraform plan`.
- `terraform destroy` will delete all the resources defined in the current configuration.
- The `terraform import` command can only import resources that are already created and exist in the target provider.
- When importing, you'll need to generate the config file of the resource you import.
- `terraform state` commands inspect the current state of the infrastructure managed by Terraform.
- Before initializing Terraform, remember to specify the provider and backend configuration in the `main.tf` file.
- The `terraform validate` command is used to validate the syntax of the Terraform files; it's a good practice to run it before applying any change.
- `terraform.tfvars` is used to store sensitive data such as credentials, and it's better not to include it in the version control.

Advanced Terraform tips and tricks

- To save the current state of the infrastructure to a file: `terraform state pull > [file_name].tfstate`
- To load the state from a file: `terraform state push [file_name].tfstate`
- To refresh the state and update it with the current infrastructure: `terraform refresh`
- To generate an execution plan and save it to a file: `terraform plan -out [file_name].plan`
- To apply an execution plan from a file: `terraform apply [file_name].plan`
- To see the current state: `terraform show`
- To return a detailed exit code to determine if changes exist between the current infrastructure and the current plan: `terraform plan -detailed-exitcode`
 - 0 = no changes; succeeded with empty diff
 - 1 = Error with plan
 - 2 = Succeeded, with changes to be made (non-empty diff)
- To generate a visual representation of either a configuration or execution plan: `terraform graph | dot -Tpng > graph.png`

Terraform best practices

- Keep the Terraform files in version control
- Use modules to organize the Terraform files
- Use variables to make the Terraform files more reusable
- Use `terraform state` commands to inspect and manage the state of the infrastructure
- Use Terraform workspaces to manage different environments (e.g. dev, staging, production)

- Use Terraform's built-in validation checks, such as `terraform validate`, to catch errors early
- Use a separate backend configuration to store the state remotely to avoid loss and conflicts.
- Use `terraform plan -detailed-exitcode` to check for changes before applying.
- Use the `terraform import` command to import existing resources and keep track of them in the state.
- Use `terraform taint` to mark a specific resource for recreation.
 - This command is useful for operational resources that are experiencing internal problems not managed by terraform.
 - For example, a VM machine may be operational, but the software running on the instance has crashed
- While `terraform state list` is excellent for seeing all infrastructure that exists, the list will be large and will be difficult to parse out information
 - Ensure that you filter by resource or by module to get back helpful information
- Use `terraform graph | dot -Tpng > graph.png` to visualize configuration or execution plans

- Terraform variable files will be executed with any name with `.tf` extension. Example → **`example.tf`**
- Terraform value file name should be **`terraform.tfvars`** for auto pick. If we change the default variable values file name from **`terraform.tfvars`** → **`demo.tfvars`** then we need to specify the file name during execution.
- **`terraform apply -var-file="demo.tfvars"`**
- If custom variable value file name can be written as **`demo.auto.tfvars`** for auto pick during execution
- When we run the **`terraform init`** command, the below file will be created.
`terraform.tfstate` **`terraform.tfstate.backup`**
- Terraform variable can also be define in the terminal (Priority will be given to the variable value provided in the terminal and it will override other variable values)
- `Terraform apply -var="instancetype=t2.micro" -var="image=ami-02a2af70a66af6dfb"`

Default data type in variable file is 'String'

AWS Project

```
1. provider "aws" {
2.   access_key = "AKIA6FUXEZWVVJ3I54B5"
3.   secret_key = "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"
4.   region     = "ap-south-1"
5. }
6.
7. # Create VPC
8.
9. resource "aws_vpc" "neelamvpc" {
10.   cidr_block = "10.0.0.0/16"
11.   instance_tenancy = "default"
12.
13.   tags = {
14.     Name = "neelamvpc"
15.   }
16. }
17.
18. #This block will create public subnet
19.
20. resource "aws_subnet" "neelam-subnet-public" {
21.   vpc_id            = aws_vpc.neelamvpc.id
22.   cidr_block        = "10.0.1.0/24"
23.   availability_zone = "ap-south-1a"
24.
25.   tags = {
26.     Name = "neelam-subnet-public"
27.   }
28. }
29.
30. # Create private subnet
31.
32. resource "aws_subnet" "neelam-subnet-private" {
33.   vpc_id            = aws_vpc.neelamvpc.id
34.   cidr_block        = "10.0.2.0/24"
35.   availability_zone = "ap-south-1a"
36.
37.   tags = {
38.     Name = "neelam-subnet-private"
39.   }
40. }
41.
42. # Create security group
43.
44. resource "aws_security_group" "neelamsg" {
45.   name            = "neelamsg"
46.   description     = "Allow TLS inbound traffic"
47.   vpc_id          = aws_vpc.neelamvpc.id
48.
49.   ingress {
```



```

50.     description      = "TLS from VPC"
51.     from_port        = 22
52.     to_port          = 22
53.     protocol         = "tcp"
54.     cidr_blocks      = ["0.0.0.0/0"]
55. }
56.
57. egress {
58.     from_port        = 0
59.     to_port          = 0
60.     protocol         = "-1"
61.     cidr_blocks      = ["0.0.0.0/0"]
62. }
63.
64. tags = {
65.     Name = "neelamsg"
66. }
67. }
68.
69.
70.
71.
72. # Create Internet Gateway
73.
74. resource "aws_internet_gateway" "neelamigw" {
75.     vpc_id = aws_vpc.neelamvpc.id
76.
77.     tags = {
78.         Name = "neelamigw"
79.     }
80. }
81.
82.
83. # Create route table
84.
85. resource "aws_route_table" "neelamrt-public" {
86.     vpc_id = aws_vpc.neelamvpc.id
87.
88.     route {
89.         cidr_block = "0.0.0.0/0"
90.         gateway_id = aws_internet_gateway.neelamigw.id
91.     }
92.
93.     tags = {
94.         Name = "neelamrt-public"
95.     }
96. }
97.
98. # Associate subnet
99.
100. resource "aws_route_table_association" "public-asso-neelam"
    {
101.     subnet_id      = aws_subnet.neelam-subnet-public.id
102.     route_table_id = aws_route_table.neelamrt-public.id
103. }
104.

```

```
105. # Create key pair
106.
107. resource "aws_key_pair" "neelam" {
108.     key_name     = "neelam"
109.     public_key   = "Enter Key"
110. }
111.
112.
113. # Create instance with public ip
114.
115. resource "aws_instance" "neelam-instance" {
116.     ami           = "ami-02a2af70a66af6dfb"
117.     instance_type = "t2.small"
118.     key_name      = "neelam"
119.     subnet_id     = aws_subnet.neelam-subnet-public.id
120.
121.     vpc_security_group_ids = [aws_security_group.neelamsg.id]
122.
123.     tags = {
124.         Name = "neelam-instance"
125.     }
126. }
127.
128.
129. # Create public ip
130.
131. resource "aws_eip" "neelam" {
132.     instance = aws_instance.neelam-instance.id
133.     vpc      = true
134. }
135.
136. # Create public ip
137.
138. resource "aws_eip" "NATip-neelam" {
139.     vpc = true
140. }
141.
142.
143. # Create NAT Gateway
144.
145. resource "aws_nat_gateway" "neelamNAT" {
146.     allocation_id = aws_eip.NATip-neelam.id
147.     subnet_id     = aws_subnet.neelam-subnet-public.id
148.
149.     tags = {
150.         Name = "neelamNAT"
151.     }
152. }
153.
154.
155. # Create route table
156.
157. resource "aws_route_table" "neelamrt-private" {
158.     vpc_id = aws_vpc.neelamvpc.id
159.
160.     route {
```

```
161.     cidr_block = "0.0.0.0/0"
162.     gateway_id = aws_nat_gateway.neelamNAT.id
163. }
164.
165.     tags = {
166.         Name = "neelamrt-private"
167.     }
168. }
```

Variable and Count Parameter

```
1. variable "instancetag" {
2.     type = list
3.     default = ["dev", "test", "prod"]
4. }
5.
6.
7. variable "instancetype" {
8.     type = list
9.     default = ["t2.nano", "t2.micro", "t2.small"]
10. }
11.
12.
13. resource "aws_instance" "neelam-instance" {
14.     ami          = "ami-02a2af70a66af6dfb"
15.     instance_type = varinstancetype[count.index]
16.     count = 3
17.
18.     tags = {
19.         Name = varinstancetag[count.index]
20.     }
21. }
```

Operators

```
variable "instancetype" {
  type = map
  default = {
    "dev" = "t2.nano",
    "test" = "t2.micro",
    "prod" = "t2.small"
  }
}

variable "image" {
  type = list
  default = ["ami-02a2af70a66af6dfb", "ami-0287a05f0ef0e9d9a",
"ami-04708942c263d8190"]
}

variable input {}

resource "aws_instance" "dev" {
  ami          = var.image[0]
  instance_type = varinstancetype["dev"]
  count = var.input == "dev" ? 1 : 0
        # other operator >=, <=, !=,

  tags = {
    Name = "dev"
  }
}

resource "aws_instance" "test" {
  ami          = var.image[1]
  instance_type = varinstancetype["test"]
  count = var.input == "test" ? 2 : 0

  tags = {
    Name = "test"
  }
}

resource "aws_instance" "prod" {
  ami          = var.image[2]
  instance_type = varinstancetype["prod"]
  count = var.input == "prod" ? 3 : 0

  tags = {
    Name = "prod"
  }
}
```

Local Variable

```
locals {
  common_tag = {
    Name = "UK-Project"
    Owner = "Rabi Ram"
  }
}

resource "aws_instance" "neelam" {
  ami          = "ami-0d13e3e640877b0b9"
  instance_type = "t2.micro"
  tags         = local.common_tag
}
```

```
resource "aws_vpc" "neelamvpc" {
  cidr_block = "10.0.0.0/16"
  instance_tenancy = "default"
  tags         = local.common_tag
}
```

Output Value

```
resource "aws_instance" "rishit" {
  ami          = "ami-0d13e3e640877b0b9"
  instance_type = "t2.micro"

  tags = {
    Name = "rishit"
  }
}

output "instance-info" {
  value = aws_instance.rishit
#value = aws_instance.rishit.ami (to print only ami of instance)
}
```

S3 Mount

```
terraform {
  backend "s3" {
    bucket      = "terra-bucket-rishit"
    key         = "state/rishit"
    region      = "ap-south-1"
    access_key  = "AKIA6FUXEZVVVJ3I54B5"
    secret_key  = "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"
    dynamodb_table = "rishit_dynamo"
  }
}

resource "aws_instance" "neelam" {
  ami          = "ami-0d13e3e640877b0b9"
  instance_type = "t2.micro"
}
```

```
tags = {
  Name = "neelam"
}
```

Terraform Workspace commands

Terraform workspaces are a powerful feature that makes it easy to manage different environments (like development, staging, and production) using the same configuration.

Subcommands:

delete	Delete a workspace
list	List Workspaces
new	Create a new workspace
select	Select a workspace
show	Show the name of the current workspace

Provisioner

Terraform Provisioners are used for executing scripts or shell commands on a local or remote machine as part of resource creation/deletion.

```
provider "aws" {
  access_key = "AKIA6FUXEZWVVJ3I54B5"
  secret_key = "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"
  region     = "ap-south-1"
}
```

```
resource "aws_key_pair" "neelam" {
  key_name    = "neelam"
  public_key = "Enter key"
}
```

```
resource "aws_instance" "neelam-instance" {
  ami           = "ami-02a2af70a66af6dfb"
  instance_type = "t2.micro"
  key_name      = "neelam"

  tags = {
    Name = "neelam-instance"
  }
}
```

```

provisioner "file" {
  source = "/root/aws/primecare-html"
  # primecare-html is name html source code folder
  destination = "/tmp"
}

provisioner "remote-exec" {
  inline = [
    "sudo yum install httpd -y",
    "sudo systemctl start httpd",
    "sudo systemctl enable httpd",
    "sudo cp -rvf /tmp/primecare-html/* /var/www/html",
    "sudo systemctl restart httpd"
  ]
}

connection {
  host = self.public_ip
  user = "ec2-user"
  type = "ssh"
  private_key = file ("./neelam")
}

```

Local Provisioner

```

provider "aws" {
  access_key = "AKIA6FUXEZWVVJ3I54B5"
  secret_key = "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"
  region     = "ap-south-1"
}

resource "aws_key_pair" "neelam" {
  key_name     = "neelam"
  public_key   = "Enter ssh key"
}

variable "privatekey" {
  default = "neelam"
}

resource "aws_instance" "neelam-instance" {
  ami          = "ami-02a2af70a66af6dfb"
  instance_type = "t2.micro"
  key_name     = "neelam"
}

```

```

tags = {
  Name = "neelam-instance"
}

provisioner "remote-exec" {
  inline = [
    "echo 'build ssh connection'"
  ]
  connection {
    host = self.public_ip
    type = "ssh"
    user = "ec2-user"
    private_key = file ("./neelam")
  }
}
provisioner "local-exec" {
  command = "ansible-playbook -i
${aws_instance.neelam-instance.public_ip}, --private-key
${var.privatekey} playbook.yml"
}
}

output "neelam_ip" {
  value = aws_instance.neelam-instance.public_ip
}

```

Packer

Packer is an open-source DevOps tool made by Hashicorp to create **identical machine images** for multiple platforms from a single JSON config file. This tool really shines when it comes to creating immutable architecture.

You can use Packer for AWS, GCP, AZURE and etc.

Packer is template-driven, templates are written in JSON format.

```

{
  "variables" : {
    "region" : ""
  },
  "builders" : [

```



```

    {
      "type" : "amazon-ecs",
      "profile" : "default",
      "region" : "{{user `region`}}",
      "instance_type" : "t2.micro",
      "source_ami" : "",
      "ssh_username" : "ec2-user"
    }
  ],
  "provisioners" : [
    {
      "type" : "shell",
      "script" : "./yourshell.sh"
    }
  ]
}

```

Importing existing infrastructure

Terraform import

`terraform import aws_s3_bucket.my_bucket rabi-terraform-bucket`

Terraform config-driven import

Create **import.tf** file

Syntax

```

import{
  id = "resource ID"
  to = resource name
}

```

`terraform plan -generate-config-out=generated.tf`

terraform apply

What is a null resource in Terraform?

The null_resource in Terraform is similar to a standard resource. It adheres to the resource lifecycle model and serves as a placeholder for executing arbitrary actions within Terraform configurations without actually provisioning any physical resources.

```
# The following null resource will print message "Hello World"
17 resource "null_resource" "null_resource_simple" {
18   provisioner "local-exec" {
19     command = "echo Hello World"
20   }
21 }
```

Terraform null_resource can be used in the following scenario -

- Run shell command
- You can use it along with local provisioner and remote provisioner
- It can also be used with Terraform Module, Terraform count, Terraform Data source, Local variables
- It can even be used for output block

What is the Terraform lifecycle?

In Terraform, the term "lifecycle" refers to a set of configurations within a Terraform resource that can influence the behaviour of that resource during creation, update, and destruction. The lifecycle block within a resource definition allows you to set certain policies that change the default behaviour of Terraform's resource management.

create_before_destroy: When set to true, this instructs Terraform to create a new resource before destroying the old one during an update.

prevent_destroy: This boolean flag, when set to true, will prevent Terraform from destroying the resource.

ignore_changes: This directive takes a list of attributes within the resource that Terraform should ignore when performing updates.

```
resource "aws_instance" "example" {
  # ...
```

```
  lifecycle {
    create_before_destroy = true
    prevent_destroy = false
    ignore_changes = [
```

```

    "ami", # Maybe the AMI is updated out of band
    "tags.Name", # Perhaps this tag is managed by another process
  ]
}
}

```

Terraform Modules

my-terraform-project/

```

├── main.tf
├── module-1/
│   ├── main.tf
│   ├── variables.tf
│   └── outputs.tf
└── module-2/
    ├── main.tf
    ├── variables.tf
    └── outputs.tf

```

EC2 Instance Module (module-1)

module-1/main.tf

```

resource "aws_instance" "this" {
  ami          = var.ami
  instance_type = var.instance_type

```

```

  tags = {
    Name = var.name
  }
}

```

module-1/variables.tf

```

hcl
variable "ami" {
  description = "The AMI to use for the instance"
  type       = string
}
variable "instance_type" {
  description = "The type of instance to create"
  type       = string
}
variable "name" {
  description = "The name of the instance"
  type       = string
}

```

module-1/outputs.tf

```

output "instance_id" {
  description = "The ID of the instance"
  value      = aws_instance.this.id
}
output "public_ip" {
  description = "The public IP of the instance"
  value      = aws_instance.this.public_ip
}

```

S3 Bucket Module (module-2)

module-2/main.tf

hcl

```

resource "aws_s3_bucket" "this" {
  bucket = var.bucket_name
  acl    = var.acl
}

```

module-2/variables.tf

```

variable "bucket_name" {
  description = "The name of the bucket"
  type       = string
}

```

```

variable "acl" {
  description = "The ACL of the bucket"
  type       = string
  default    = "private"
}

```

module-2/outputs.tf

```

output "bucket_arn" {
  description = "The ARN of the bucket"
  value      = aws_s3_bucket.this.arn
}
output "bucket_name" {
  description = "The name of the bucket"
  value      = aws_s3_bucket.this.bucket
}

```

Main Configuration (main.tf)

Now, create a main.tf file to call these modules.

main.tf

```

provider "aws" {
  region = "us-west-2"
}

```

```

module "ec2_instance" {

```

```
source      = ". /module-1"
```

Below lines are optional

```
ami          = "ami-0c55b159cbfafe1f0" # Replace with a valid AMI ID for your region
instance_type = "t2.micro"
name         = "example-instance"
}
```

```
module "s3_bucket" {
  source      = ". /module-2"
  bucket_name = "my-example-bucket-123456789"
  acl        = "private"
}
output "instance_id" {
  value = module.ec2_instance.instance_id
}
output "instance_public_ip" {
  value = module.ec2_instance.public_ip
}
```

Explicit Dependencies

```
resource "aws_security_group" "example"
```

```
resource "aws_instance" "example"
```

Explicit Dependencies

You can explicitly define dependencies using the `depends_on` argument when Terraform cannot infer dependencies automatically, or you need to enforce a specific order of resource creation.

Example:

```
resource "aws_instance" "example" {
  ami          = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"
  depends_on = [aws_security_group.example]
}
```

```
resource "aws_security_group" "example" {
  name        = "example"
  description = "Example security group"
}
```

Module Dependencies

```
module "vpc" {
  source = ". /modules/vpc"
  # VPC module outputs a VPC ID
```

```
}
```

```
module "ec2" {  
  source = "./modules/ec2"  
  vpc_id = module.vpc.vpc_id  
  
  depends_on = [module.vpc]  
}
```